

```
# Drawing API Upgrade Guide (D101664915)
> **Directive for the LLM applying these changes:** Make sure to apply not
just to the code, but to also examine and update the md files in
Assistant/skills as necessary.
This guide upgrades consumers of the procedural Drawing API after
D101664915, which promotes `DrawingCommandsBuilder`, `DrawingCommandData`,
the brush/pen/font resources, and the drawing-specific enums from
`meta/custom_ui_experimental` to durable `meta/custom_ui` (and re-exports
them through the durable `worlds_sdk` kit / `meta/worlds`).
The promotion folds in the RFC v3 shape changes, so call sites must be
updated in the same pass as the import-path swap.
```

```
---
```

### ## 1. Update import paths

| Symbol                                                                                                                                                                                                                                                                                                                                                                          | Old import                                                                                         | New import                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>`DrawingCommandsBuilder`</code> , <code>`DrawingCommandData`</code> , <code>`Brush`</code> , <code>`SolidBrush`</code> , <code>`LinearGradientBrush`</code> , <code>`RadialGradientBrush`</code> , <code>`ImageBrush`</code> , <code>`Pen`</code> , <code>`Font`</code> , <code>`GradientStop`</code> , <code>`DrawTextOptions`</code> , <code>`ImageBrushOptions`</code> | <code>`meta/custom_ui_experimental`</code>                                                         | <code>`meta/custom_ui`</code> (in-tree) <b>**or**</b> <code>`meta/worlds`</code> (Worlds SDK consumers)                                               |
| <code>`BrushType`</code> (NEW enum, see §3)                                                                                                                                                                                                                                                                                                                                     | n/a                                                                                                | <code>`meta/custom_ui`</code> / <code>`meta/worlds`</code>                                                                                            |
| <code>`FontFamily`</code> , <code>`FontWeight`</code> , <code>`FontStyle`</code> , <code>`FontStretch`</code> , <code>`Stretch`</code> , <code>`TileMode`</code> , <code>`CoordinateType`</code> , <code>`DrawFlowDirection`</code> , <code>`DrawTextAlignment`</code> , <code>`DrawTextTrimming`</code>                                                                        | <code>`meta/custom_ui_experimental`</code>                                                         | <code>`meta/custom_ui`</code> / <code>`meta/worlds`</code>                                                                                            |
| <code>`IRectangle`</code> , <code>`IVec2`</code>                                                                                                                                                                                                                                                                                                                                | <code>`meta/platform_api`</code> ( <code>`IRectangle`</code> ) / inferred ( <code>`IVec2`</code> ) | <code>`meta/custom_ui`</code> / <code>`meta/worlds`</code> (also still exported from <code>`meta/platform_api`</code> for <code>`IRectangle`</code> ) |

#### **\*\*Rules:\*\***

- 1P/2P consumers inside the engine: prefer ``meta/custom_ui``.
- 3P / Worlds SDK consumers: import from ``meta/worlds`` (the ``worlds_sdk`` kit re-exports the full surface).
- Do **\*\*not\*\*** import drawing types from ``meta/custom_ui_experimental`` anymore. The experimental kit re-exports of the drawing surface have been removed; only ``CustomUiApiExperimental.fetchElementBounds`` and its ``CustomUiExperimentalService`` wrapper remain experimental.

### ### Generic Noesis enums – important caveat

The generic Noesis enums ``Visibility``, ``HorizontalAlignment``, ``VerticalAlignment``, ``Orientation``, ``ScrollBarVisibility``, ``TextAlignment``, ``TextWrapping``, ``TextTrimming``, ``FlowDirection`` moved with the file into ``meta/custom_ui``, **\*\*but are intentionally NOT re-exported from ``worlds_sdk`` / ``meta/worlds``\*\*** to avoid a ``TextAlignment`` symbol collision with ``meta/core_ui_experimental``.

- In-tree (``meta/custom_ui``): import them from ``meta/custom_ui``.
- Worlds SDK consumers: keep importing them from wherever you imported them before (typically ``meta/core_ui_experimental`` or

`meta/custom\_ui\_experimental` if you were a 1P consumer pre-promotion).  
The worlds\_sdk surface deliberately omits them.

---

## 2. Signature changes – replace positional scalars with `IRectangle` / `IVec2`

`Rectangle` and `Vec2` from `meta/platform\_api` are structurally assignable to `IRectangle` and `IVec2`, so existing `new Rectangle(...)` / `new Vec2(...)` instances continue to work. Plain object literals (`{x, y, width, height}` or `{x, y}`) also work.

### 2.1 `drawRect` – unchanged

Already takes `IRectangle`. No call-site change.

### 2.2 `drawRoundRect` – `(rx, ry)` → `radii: IVec2`

```ts

// Before

builder.drawRoundRect(brush, pen, rect, 8, 12);

// After

builder.drawRoundRect(brush, pen, rect, new Vec2(8, 12));

// or: builder.drawRoundRect(brush, pen, rect, {x: 8, y: 12});

```

### 2.3 `drawEllipse` – `(cx, cy, rx, ry)` → `(center: IVec2, radii: IVec2)`

```ts

// Before

builder.drawEllipse(brush, pen, 50, 60, 30, 40);

// After

builder.drawEllipse(brush, pen, new Vec2(50, 60), new Vec2(30, 40));

```

### 2.4 `drawText` – `(x, y, w, h)` → `rect: IRectangle`

```ts

// Before

builder.drawText('Hi', 0, 0, 100, 50, 16, brush, font);

builder.drawText('Hi', 0, 0, 100, 50, 16, brush, font, options);

// After

builder.drawText('Hi', new Rectangle(0, 0, 100, 50), 16, brush, font);

builder.drawText('Hi', new Rectangle(0, 0, 100, 50), 16, brush, font,

options);

// or: builder.drawText('Hi', {x: 0, y: 0, width: 100, height: 50}, 16, brush, font);

```

### 2.5 `drawImage` – `(x, y, w, h)` → `rect: IRectangle`

```ts

// Before

builder.drawImage(image, 5, 10, 100, 200);

// After

builder.drawImage(image, new Rectangle(5, 10, 100, 200));

```

### 2.6 `pushTranslate` – `(x, y)` → `offset: IVec2`

```

```ts
// Before
builder.pushTranslate(15, 25);
// After
builder.pushTranslate(new Vec2(15, 25));
```

### 2.7 `pushRotate` — `(angleDeg, cx, cy)` → `(angleDeg, center: IVec2)`
```ts
// Before
builder.pushRotate(45, 100, 200);
// After
builder.pushRotate(45, new Vec2(100, 200));
```

### 2.8 `pushScale` — `(sx, sy, cx, cy)` → `(scale: IVec2, center: IVec2)`
```ts
// Before
builder.pushScale(2, 3, 50, 60);
// After
builder.pushScale(new Vec2(2, 3), new Vec2(50, 60));
```

### 2.9 `drawLine`, `LinearGradientBrush`, `RadialGradientBrush`
The runtime shape did not change — these already took 2D points. The parameter type widened from `Vec2` to `IVec2`. Existing `new Vec2(...)` arguments continue to work; plain object literals are now also accepted.

### 2.10 `ImageBrushOptions.sourceRect`
Type is now `IRectangle` (was a separate `(x, y, w, h)` tuple in some pre-RFC drafts; if your code already passed an object, no change is needed). Pass a `Rectangle` instance or `{x, y, width, height}`.

---
## 3. `Brush.type` is now an enum, not a string literal
```ts
// Before
abstract readonly type: 'solid' | 'linearGradient' | 'radialGradient' | 'image';
// After
import {BrushType} from 'meta/custom_ui'; // or 'meta/worlds'
abstract readonly type: BrushType;
```

Update any consumer that switches on `brush.type`:
```ts
// Before
switch (brush.type) {
  case 'solid': /* ... */ break;
  case 'linearGradient': /* ... */ break;
  case 'radialGradient': /* ... */ break;
  case 'image': /* ... */ break;
}

```

```
// After
switch (brush.type) {
  case BrushType.Solid: /* ... */ break;
  case BrushType.LinearGradient: /* ... */ break;
  case BrushType.RadialGradient: /* ... */ break;
  case BrushType.Image: /* ... */ break;
}
...

Numeric values are stable: `Solid = 0`, `LinearGradient = 1`,
`RadialGradient = 2`, `Image = 3`.
---
```

#### ## 4. Known consumers that must be migrated

Per the diff's "Known follow-ups" list, these still use the old import path and pre-RFC signatures:

```
- `arvr/assets/hsr_packages/kakuro_puzzle/scripts/KakuroGameComponent.ts`
-
`arvr/projects/buzzcode_investigations/examples/networked-grid-games/scripts/GameLobbyClient.ts`
-
`arvr/projects/buzzcode_investigations/examples/networked-grid-games/scripts/GridRenderer.ts`
-
`arvr/projects/buzzcode_investigations/examples/networked-grid-games/scripts/MinesweeperClient.ts`
```

Apply \$1-\$3 to each of these files.

When migrating, also search the surrounding folder for sibling scripts importing from `meta/custom\_ui\_experimental` for any drawing symbol – the experimental re-exports of the drawing API have been removed, so any remaining import will fail to resolve.

#### ## 5. Per-file migration checklist

For each consumer file:

1. Replace the `from 'meta/custom\_ui\_experimental'` clause with:
  - `from 'meta/custom\_ui'` for in-tree code, **\*\*or\*\***
  - `from 'meta/worlds'` for Worlds SDK / 3P scripts.
2. If the file imports any of `Visibility`, `HorizontalAlignment`, `VerticalAlignment`, `Orientation`, `ScrollBarVisibility`, `TextAlignment`, `TextWrapping`, `TextTrimming`, `FlowDirection`: keep the original import source for SDK consumers (these are NOT exported from `meta/worlds`).
3. Update every `drawText`, `drawImage`, `drawRoundRect`, `drawEllipse`, `pushTranslate`, `pushRotate`, `pushScale` call to pass `IRectangle` / `IVec2` (see §2).
4. Update any `brush.type` comparison to use `BrushType.\*` (see §3).
5. Re-run TypeScript validation: `rl --worlds codegen` (regenerates `index.d.ts`, `typescript\_api\_index.jsonc`, `aliased\_symbol\_manifest.jsonc`).

6. Re-run unit tests for the touched module:

```
```\n\n    buck2 test @arvr/projects/mhe/mode/linux/development\n    //path/to/module:unit_tests\n    ```
```

7. For changes inside ``arvr/apps/worlds/frameworks/custom_ui/...``, also run:

```
```\n\n    buck2 test @arvr/projects/mhe/mode/linux/development\n    //arvr/apps/worlds/frameworks/custom_ui:unit_tests --\n    DrawingCommandsBuilderTests\n    ```
```

---

## 6. Skill / `.md`` file audit (DO NOT SKIP)

Per the directive at the top of this document, after applying the code changes, search every ``Assistant/skills/**/*md`` file (and any other agent-facing `.md`` documentation that references the Drawing API) and update:

- `**Import path examples**` – replace ``from 'meta/custom_ui_experimental'`` with ``from 'meta/custom_ui'`` (in-tree) or ``from 'meta/worlds'`` (SDK).
- `**Code snippets**` showing ``drawText(text, x, y, w, h, ...)``, ``drawImage(image, x, y, w, h)``, ``drawRoundRect(..., rx, ry)``, ``drawEllipse(..., cx, cy, rx, ry)``, ``pushTranslate(x, y)``, ``pushRotate(angleDeg, cx, cy)``, ``pushScale(sx, sy, cx, cy)`` – rewrite to use ``IRectangle`` / ``IVec2`` per §2.
- `**`brush.type` comparisons**` – replace string literals with ``BrushType.*`` per §3.
- `**API stability tags / "experimental" warnings**` – the Drawing API is now durable. Remove "experimental", "subject to change", or "not in worlds\_sdk" disclaimers tied to the drawing surface; keep them only for ``CustomUiApiExperimental.fetchElementBounds`` / ``CustomUiExperimentalService``, which remain experimental.
- `**Symbol availability tables**` – add ``DrawingCommandsBuilder``, ``DrawingCommandData``, brush/pen/font types, and the drawing enums to any "available in worlds\_sdk" list. Note the Noesis-generic enums (``Visibility``, ``HorizontalAlignment``, etc.) are still NOT in ``worlds_sdk``. Recommended search patterns when auditing skill files (use the ``search_files`` MCP tool, not bash grep):
- ``meta/custom_ui_experimental`` (any reference to drawing types from this path is now stale)
- ``DrawingCommandsBuilder``
- ``drawText`s*`(``, ``drawImage`s*`(``, ``drawRoundRect`s*`(``, ``drawEllipse`s*`(``
- ``pushTranslate`s*`(``, ``pushRotate`s*`(``, ``pushScale`s*`(``
- ``brush.type`` and the literal strings ``'solid'``, ``'linearGradient'``, ``'radialGradient'``, ``'image'`` in a drawing context

---

## ## 7. Reference

- Diff: D101664915 - `[MHE][CustomUI] Promote procedural Drawing API to durable`
- Implementation:  
`arvr/apps/worlds/frameworks/custom\_ui/assets/scripts/drawing/DrawingCommands.ts`
- Re-exports:  
`arvr/apps/worlds/frameworks/custom\_ui/assets/scripts/index.ts`,  
`arvr/projects/mhe/kits/worlds\_sdk/index.ts`
- Type declarations: `arvr/projects/mhe/kits/worlds\_sdk/index.d.ts`
- Tests (canonical migration examples):  
`arvr/apps/worlds/frameworks/custom\_ui/assets/tests/DrawingCommandsBuilderTests.ts`